

Manual de trabajo: pc-diagnostico

Guía práctica (paso a paso) para trabajar con tu proyecto **Tauri + Vite/React + Rust** usando **VS Code** y **GitHub**, con flujo profesional (**dev/main**), CI y Releases automáticas (MSI + EXE).

Repo: **cayu007/pc-diagnostico** • Fecha de referencia: **2026-03-02**

Resumen rápido del flujo

- Trabaja siempre en **dev** (o en **feature/*** si es una tarea grande).
- Sube cambios a GitHub con **commit + push**.
- Para publicar: PR **dev** → **main** + merge.
- Para sacar instaladores: crea un **tag (vX.Y.Z)** en **main**. GitHub compila y sube **.msi** y **.exe** a Releases.

Contenido

1. Configuración inicial (branch protection + CI)
2. Trabajo diario desde VS Code (local)
3. Cambios hechos en GitHub (web/Codex) y cómo traerlos
4. Pasar cambios a main (Pull Request) y publicar versión
5. Cómo comprobar si todo está sincronizado
6. Problemas típicos y soluciones
7. Chuleta de comandos

1. Configuración inicial

1.1 Proteger la rama main (obligar PR y CI verde)

Como tu repo es **público**, puedes forzar que no haya push directo a **main**. Así evitas romper producción y te obligas a pasar por PR + CI.

En GitHub:

- 1 Repo → **Settings** → **Branches** → **Add branch protection rule**
- 2 Branch name pattern: **main**
- 3 Activa: **Require a pull request before merging**
- 4 Activa: **Require status checks to pass before merging**
- 5 Marca el check del workflow CI (normalmente **CI / build-windows**)
- 6 Activa: **Require branches to be up to date before merging**
- 7 Save changes

Si no te aparece el check para seleccionarlo, haz primero un push a dev para que el workflow CI se ejecute al menos una vez. Luego vuelve a la regla y ya saldrá.

1.2 Workflows: CI vs Release

CI (ci.yml) se ejecuta en cada push/PR para validar que compila. **Release (release.yml)** se ejecuta solo cuando creas un tag **vX.Y.Z** y publica instaladores (MSI/EXE).

2. Trabajo diario desde VS Code (local)

2.1 Antes de empezar a trabajar

Asegúrate de estar en **dev** y traer lo último:

```
git checkout dev
git pull
```

Comprueba el estado:

```
git status
```

2.2 Hacer cambios y subirlos (rutina)

```
git status
git add .
git commit -m "Feat: descripcion corta"
git push
```

Consejo: commits pequeños, mensajes claros (Feat/Fix/Chore).

2.3 Cambios grandes: ramas feature/*

```
git checkout dev
git pull
git checkout -b feature/nombre-corto
# ... trabajas ...
git add .
git commit -m "Feat: ..."
git push -u origin feature/nombre-corto
```

Luego abres PR: **feature/*** → **dev**.

3. Cambios en GitHub (web/Codex) y cómo traerlos

3.1 Traer a local lo que cambió en GitHub

```
git fetch
git status
# Si dice 'behind', haz:
git pull
```

Hazlo en la rama correcta (dev/main/feature).

3.2 Conflictos al hacer pull

Si tú y GitHub tocasteis las mismas líneas, resuelve en VS Code y luego:

```
git add .
git commit -m "Fix: resolve merge conflict"
git push
```

4. Pasar cambios a main y publicar versión

4.1 PR dev → main

- 1 GitHub → Pull requests → New pull request
- 2 Base: **main** • Compare: **dev**
- 3 Espera CI verde → Merge

Después en tu PC:

```
git checkout main
git pull
```

4.2 Release automática (MSI + EXE) con tags

```
git checkout main
git pull
git tag v0.1.0
git push --tags
```

Después: GitHub → Actions (ver build) → Releases (descargas .msi y .exe).

5. Comprobar si todo está sincronizado

```
git fetch
git status
```

- **behind:** te falta pull
- **ahead:** te falta push
- **diverged:** pull + resolver conflictos

6. Problemas típicos y soluciones rápidas

Problema	Síntoma	Solución (rápida)
Push rechazado	rejected / failed to push	git pull --rebase (git add/commit si hay conflictos) git push
Conflictos	CONFLICT ...	Resuelve → git add . → git commit → git push
CI falla	Actions rojo	Revisa Actions. En local: npm ci + npm run tauri build
Falta espacio	os error 112	Borra src-tauri/target o (cd src-tauri) cargo clean
No veo cambios	local desfasado	git fetch; git status; git pull

7. Chuleta de comandos

Objetivo	Comando
Ver estado	git status
Cambiar a dev	git checkout dev
Actualizarte	git pull
Subir cambios	git add . && git commit -m "..." && git push
Crear feature	git checkout -b feature/nombre
Crear tag release	git tag v0.1.0 && git push --tags